

Bases de Datos NoSQL: Diferencias, Ventajas y Desventajas

1. Key-Value Stores

Características Principales

Características de las Claves:

- A diferencia de las claves RDBMS, estas pueden ser números, strings, JSON, tipos binarios poco comunes como imágenes, o incluso conjuntos o listas
- No existe método para escanear o buscar valores, por lo que la estrategia de nomenclatura de claves es crucial
- Las claves deben ser significativas y elegidas cuidadosamente considerando los límites de agregación
- El diseño de nomenclatura debe considerar datos específicamente dirigidos, no solo secuencias aleatorias

Características de los Valores:

- Aumenta la eficiencia de escritura y lectura, limitación de valores y reducción de latencia
- La agregación grande es una práctica común, a diferencia de RDBMS
- Estrategia estándar: mantener junta toda la información comúnmente utilizada
- Los valores pueden ser estructurados o no estructurados sin valores por defecto y restricciones
- Los valores pequeños soportan buffer de memoria (cache)

Ventajas

Simplicidad: Flexibilidad total en el modelado de datos sin estructura impuesta

Alto rendimiento y velocidad: No requiere búsquedas complejas, solo operaciones get/put/delete

Escalabilidad: Maneja bien el tamaño y las operaciones son independientes

Capacidades de cache: Mantiene datos frecuentemente usados en memoria

Experiencia de usuario personalizada: Maneja grandes volúmenes de datos de comportamiento

Respuesta rápida de aplicaciones: Recuperación inmediata de datos

Portabilidad y rentabilidad: Migración fácil entre sistemas

Desventajas

Recuperación limitada: Solo se puede acceder a la información mediante la clave

Falta de consistencia: No mantiene consistencia en transacciones múltiples

Gestión de claves compleja: Crear millones de claves únicas se vuelve desafiante

2. Document-Oriented Databases

Características Principales

- Soportan índices secundarios, permitiendo búsquedas eficientes por campos dentro de los documentos
- Los documentos pueden almacenar datos en formatos como JSON, XML o BSON, usando estructuras jerárquicas y auto-descriptivas
- Permiten acceso a datos mediante HTTP o Apache Thrift, facilitando la interoperabilidad entre lenguajes
- No requieren que todos los documentos tengan la misma estructura, permitiendo agregar información nueva sin modificar el esquema global
- Cada documento tiene un identificador único para operaciones de consulta, modificación y eliminación
- Utilizan algoritmos como Map-Reduce para agregación de datos

Ventajas

- **Flexibilidad de esquema:** No es necesario definir la estructura de los datos por adelantado, permitiendo crecer y modificar el diseño fácilmente
- **Almacenamiento dinámico:** Permiten guardar grandes volúmenes de datos en formatos variados
- **Facilidad de desarrollo:** El modelado de datos es intuitivo y se adapta bien a objetos en código, evitando particiones complejas y Joins
- **Consultas potentes:** Ofrecen indexación flexible y consultas ad hoc sobre colecciones de documentos
- **Escalabilidad y evolución:** Las aplicaciones pueden evolucionar sin necesidad de migraciones de esquema

Desventajas

- **Relaciones complejas:** Es difícil crear relaciones entre documentos, lo que puede complicar el diseño de datos relacionados
- **Referencias limitadas:** Las referencias entre documentos no funcionan tan bien como en bases relacionales y pueden ser frustrantes
- **Gestión de documentos:** Administrar múltiples documentos puede ser desafiante, especialmente en operaciones masivas
- **Agregaciones imprecisas:** Las operaciones de agregación pueden no ser tan precisas o eficientes como en otros modelos

3. Wide-Column Stores

Características Principales

- También conocidas como column family database, column-oriented database, wide-column store, columnar database y columnar store
- Agrupan datos similares en columnas y permiten mapas multidimensionales y anidados, donde la información se organiza en familias de columnas
- Al igual que las bases relacionales, almacenan datos en filas y columnas, pero pueden manejar tipos de datos más complejos y ambiguos, como texto sin formato e imágenes
- Todas las familias de columnas se almacenan en un keyspace, que es el agrupamiento externo de datos
- Cada familia de columnas contiene varias filas, y cada fila puede tener muchas columnas. Los datos se almacenan como pares nombre/valor con timestamp, y físicamente se guardan por familia de columna
- La estructura de una familia de columnas es flexible, permitiendo agregar o eliminar columnas en tiempo de ejecución
- Algunas bases wide-column permiten anidar familias de columnas (super column families)
- Ofrecen interfaces de cliente dinámicas, con capacidades de indexación y consulta por clave de fila, nombre de familia de columna, columna específica o timestamp

Ventajas

- Funcionan bien con datos organizados y semi-estructurados, y las actualizaciones son sencillas
- Facilitan la exploración de datos y ofrecen mejor compresión que los sistemas basados en filas, lo que reduce el almacenamiento necesario para índices, vistas y agregaciones
- Permiten almacenar grandes cantidades de información en una sola columna y mantener alto rendimiento
- Son muy eficientes en consultas de agregación (SUM, COUNT, AVG), ideales para proyectos con alto volumen de consultas en poco tiempo
- Al consultar por atributos específicos, se revisan menos bloques de disco, acelerando las búsquedas
- Altamente escalables y fáciles de distribuir en grandes clústeres de máquinas, permitiendo almacenar y procesar grandes volúmenes de datos
- Los usuarios pueden agregar nuevas columnas sin afectar el funcionamiento general, ya que no hay restricción de que todas las filas tengan las mismas columnas

Desventajas

- Insertar y eliminar datos de un solo registro puede ser más lento y requiere recorrer varias columnas
- Actualizar múltiples atributos de un registro es costoso en términos de recursos, mientras que en RDBMS es más eficiente

- Las búsquedas o consultas que requieren unir información de varias columnas pueden tener un rendimiento inferior
- No son adecuadas para aplicaciones OLTP, ya que requieren muchas lecturas y escrituras en numerosas columnas por cada elemento

4. Graph Databases

Características Principales

- Utilizan almacenamiento nativo de grafos, diseñado específicamente para gestionar y almacenar estructuras de grafos
- Algunas bases de datos serializan los grafos en sistemas relacionales, orientados a objetos u otros almacenes generales, pero las bases nativas ofrecen mejor rendimiento
- Cuentan con capacidades de procesamiento nativas, donde cada nodo tiene acceso directo a sus vecinos sin necesidad de índices globales, lo que agiliza las consultas locales
- Permiten consultas basadas en el modelo de grafo, como localizar nodos, encontrar vecinos en uno o varios saltos, escanear aristas y extraer atributos
- Las consultas más avanzadas incluyen búsquedas de subgrafos y supergrafos, que pueden requerir cálculos iterativos
- El modelo de grafo facilita la representación de relaciones complejas y permite agregar conexiones, etiquetas, subgrafos y nodos sin afectar la funcionalidad existente

Ventajas

- **Alto rendimiento en datos relacionados:** Las consultas sobre datos conectados mantienen su velocidad incluso cuando el tamaño del grafo crece, a diferencia de las bases relacionales donde los Joins se vuelven más lentos
- **Flexibilidad y evolución:** Es fácil agregar nuevas relaciones y nodos sin afectar el funcionamiento de la aplicación, ideal para desarrollos ágiles y necesidades cambiantes
- **Integración sencilla:** Permiten integrar datos de distintas fuentes sin necesidad de modificar esquemas, facilitando la gestión de datos semánticos complejos
- **Consultas eficientes:** El tiempo de ejecución de las consultas depende del tamaño del subgrafo visitado, no del tamaño total de la base de datos

Desventajas

- **Limitaciones en estructura y restricciones:** La definición de estructuras y restricciones puede ser limitada, lo que puede generar inconsistencias en los datos
- **Escalabilidad horizontal compleja:** Dividir un grafo para distribuirlo eficientemente entre varios nodos es difícil, y muchas bases de datos no soportan consultas declarativas ni optimización avanzada

- **Consultas costosas en grafos grandes:** Algunas operaciones, como el análisis global o consultas sobre grafos dinámicos, pueden ser costosas y difíciles de optimizar
- **Gestión de historial:** Almacenar y consultar el historial de cambios en grafos grandes puede ser complejo y requerir soluciones especializadas